ЛАБОРАТОРНА РОБОТА №4

Тема: CRUD-операції в EF Core. Валідація даних.

Мета: набути навичок роботи з ORM, навчитися реалізовувати операції створення, читання, оновлення та видалення даних з використанням EF Core, набути навичок роботи з механізмами валідації даних.

Хід роботи:

Завдання 1.

Використовуючи підхід Code First, створити модель, яка разом з існуючою моделлю, утворюватиме зв'язок один-до-багатьох.

Лістинг Vacancy.cs:

```
using System.ComponentModel.DataAnnotations;
using System.Collections.Generic;

namespace Lab4.Models
{
    public class Vacancy
    {
        public long VacancyId { get; set; }

        [Required(ErrorMessage = "Please enter a title")]
        [StringLength(100, ErrorMessage = "Title cannot be longer than 100 characters")]
        public string Title { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please enter a description")]
        public string Description { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please enter a company name")]
 [StringLength(50, ErrorMessage = "Company name cannot be longer than 50 characters")]
        public string Company { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please enter a salary")]
        [Range(0.01, double.MaxValue, ErrorMessage = "Please enter a positive salary")]
        public decimal Salary { get; set; }

        [Required(ErrorMessage = "Please specify a category")]
        [StringLength(50, ErrorMessage = "Category cannot be longer than 50 characters")]
        public string Category { get; set; } = string.Empty;

        public ICollection<Lab4.Models.Application> Applications { get; set; } = new
List<Lab4.Models.Application>(); // Зв'язок "один-до-багатьох"

    }
}
```

Лістинг Application.cs:

```
using System.ComponentModel.DataAnnotations;

namespace Lab4.Models
{
    public class Application
    {
        public long ApplicationId { get; set; }

        [Required(ErrorMessage = "Candidate ID is required")]
        public string CandidateId { get; set; } = string.Empty;

        public long VacancyId { get; set; }

        [Required(ErrorMessage = "Please enter a resume")]
      [StringLength(1000, ErrorMessage = "Resume cannot be longer than 1000 characters")]
        public string Resume { get; set; } = string.Empty;

        public DateTime AppliedDate { get; set; }

        public Vacancy Vacancy { get; set; }
    }
}
```

Лістинг JobDbContext.cs:

```
using Microsoft.EntityFrameworkCore;

namespace Lab4.Models
{
    public class JobDbContext : DbContext
    {
        public JobDbContext(DbContextOptions<JobDbContext> options) : base(options)
        {
        }
        public DbSet<Vacancy> Vacancies { get; set; }
        public DbSet<Application> Applications { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Lab4.Models.Application>()
                .HasOne(a => a.Vacancy)
                .WithMany(v => v.Applications)
                .HasForeignKey(a => a.VacancyId);
        }
    }
}
```

Завдання 2.

Реалізувати CRUD (Create, Read, Update, Delete) операції для обох моделей, враховуючи наступні вимоги:

- Реалізувати представлення з формами для операцій Create та Update

Рис. 1. Сторінка додавання вакансії.

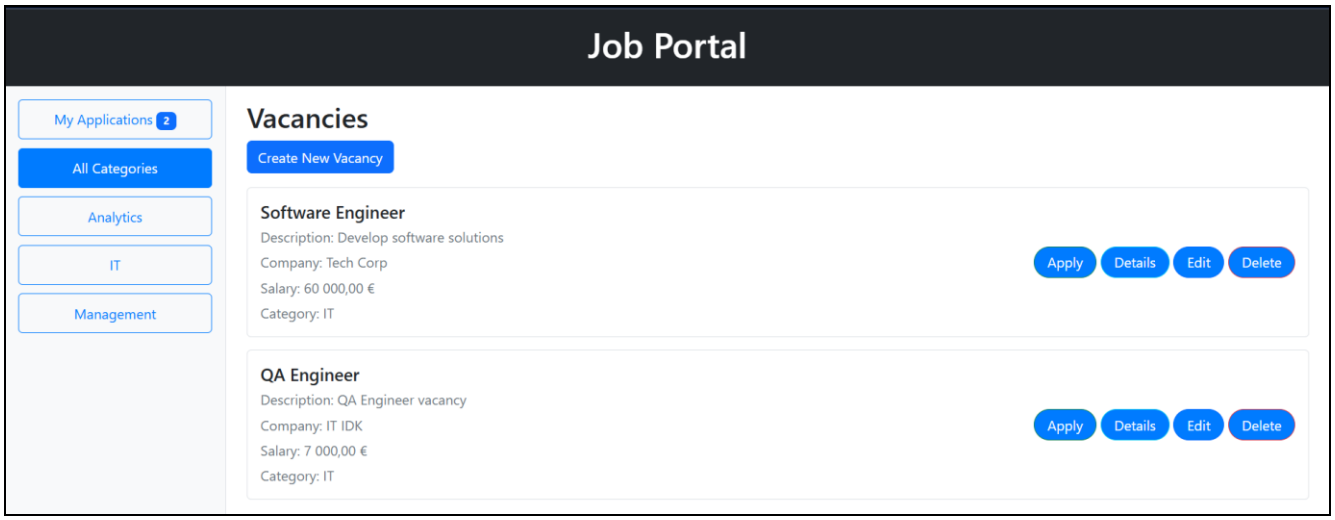

Рис. 2. Результат додавання вакансії.

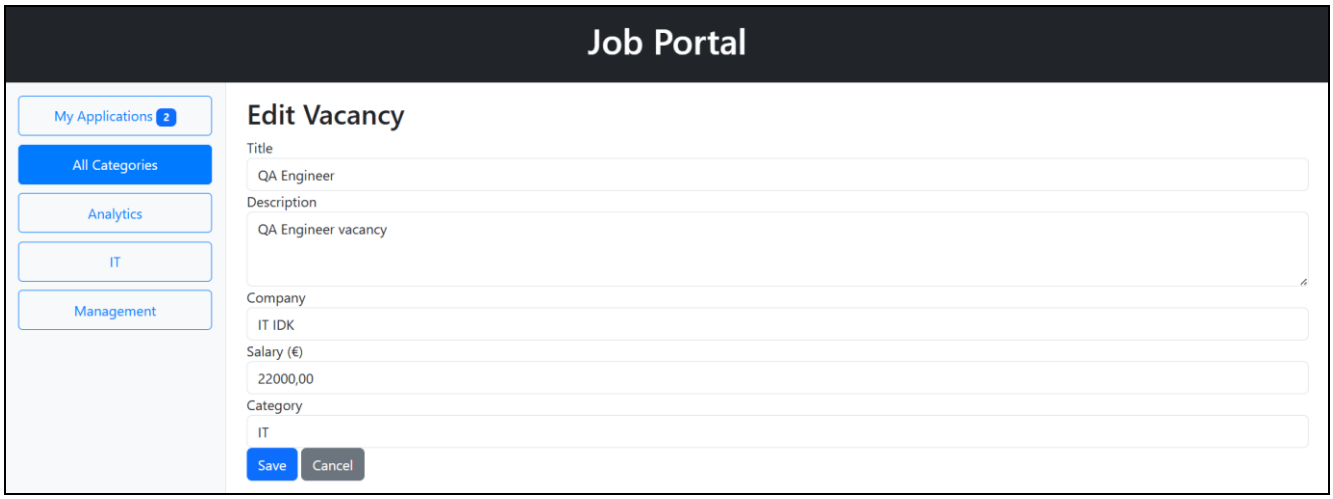

Рис. 3. Сторінка редагування вакансії.

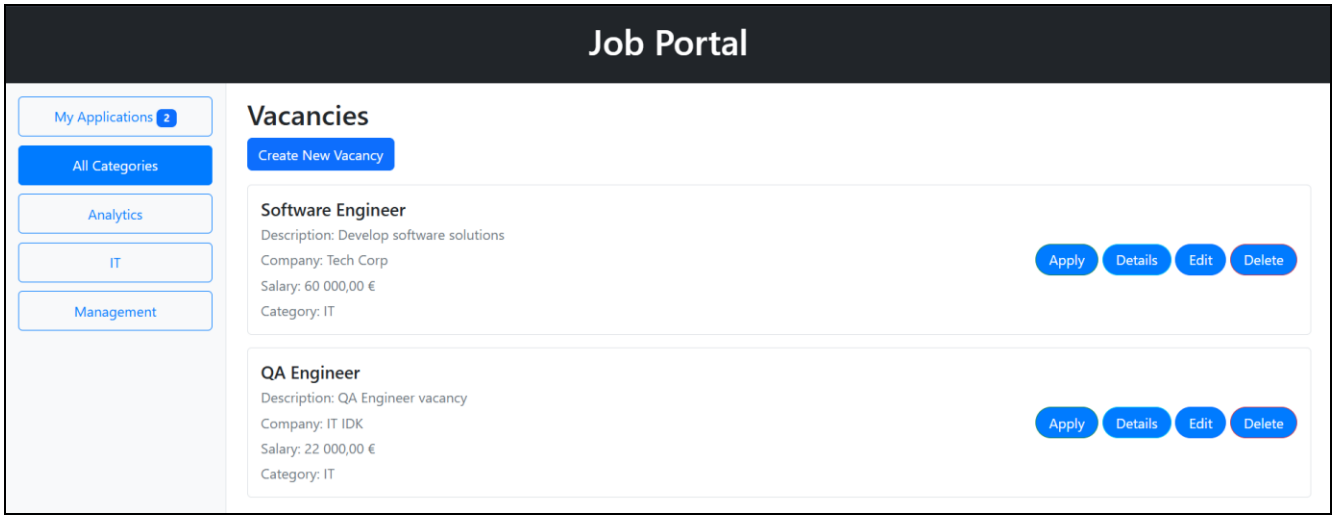

Рис. 4. Результат редагування вакансії.

- Для виконання операції Read реалізувати окрему Details сторінку, на якій будуть виводитися всі дані про сутність.

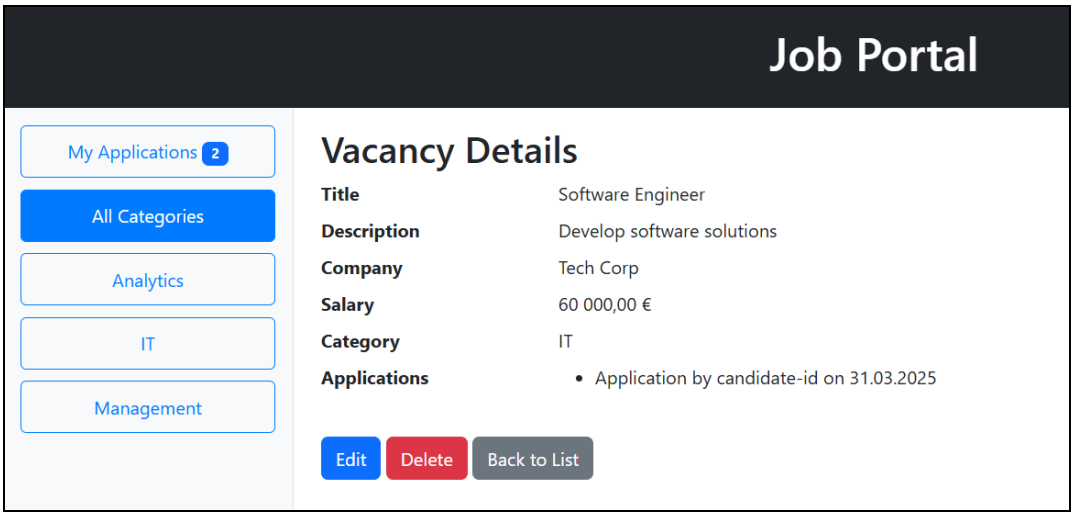

Рис. 5. Сторінка з деталями про вакансію.

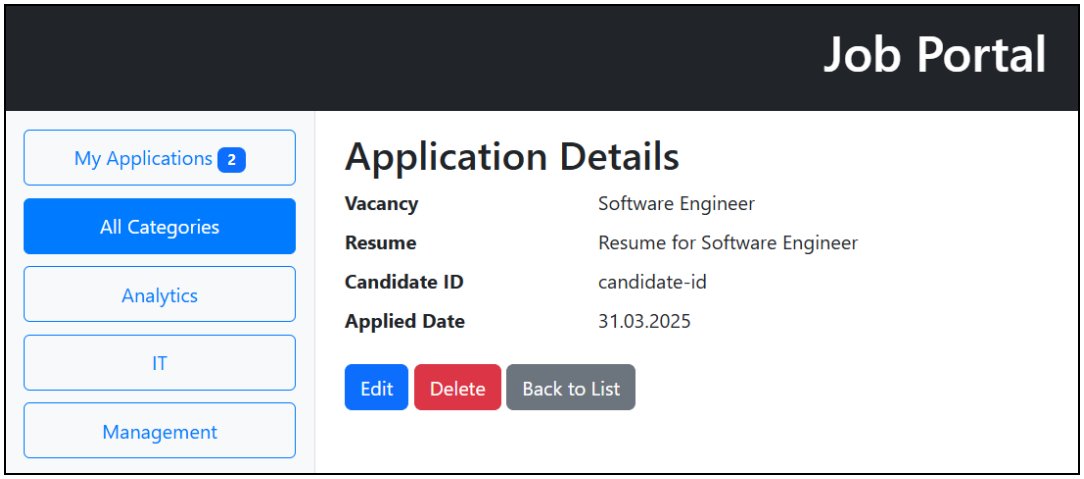

Рис. 6. Сторінка з деталями про заявку.

- Перед виконанням операції Delete запитати користувача, підтвердження видалення запису.

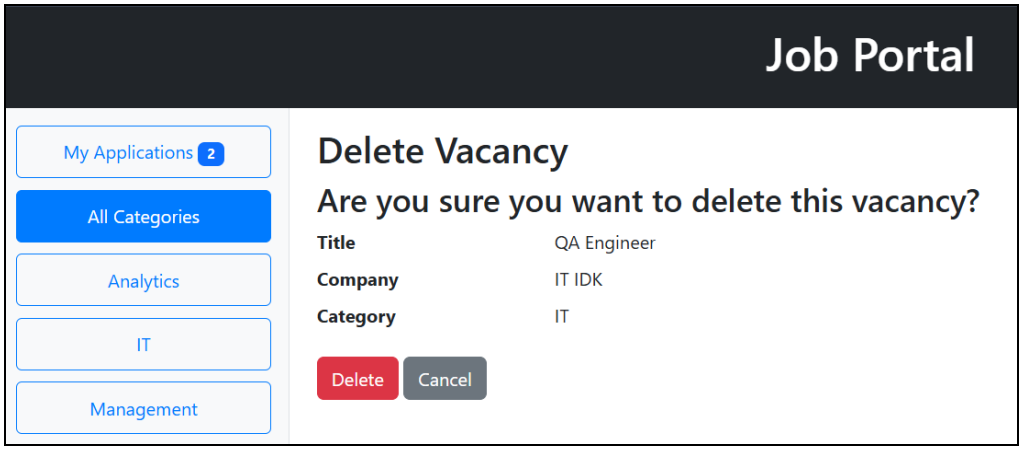

Рис. 7. Підтвердження видалення вакансії.

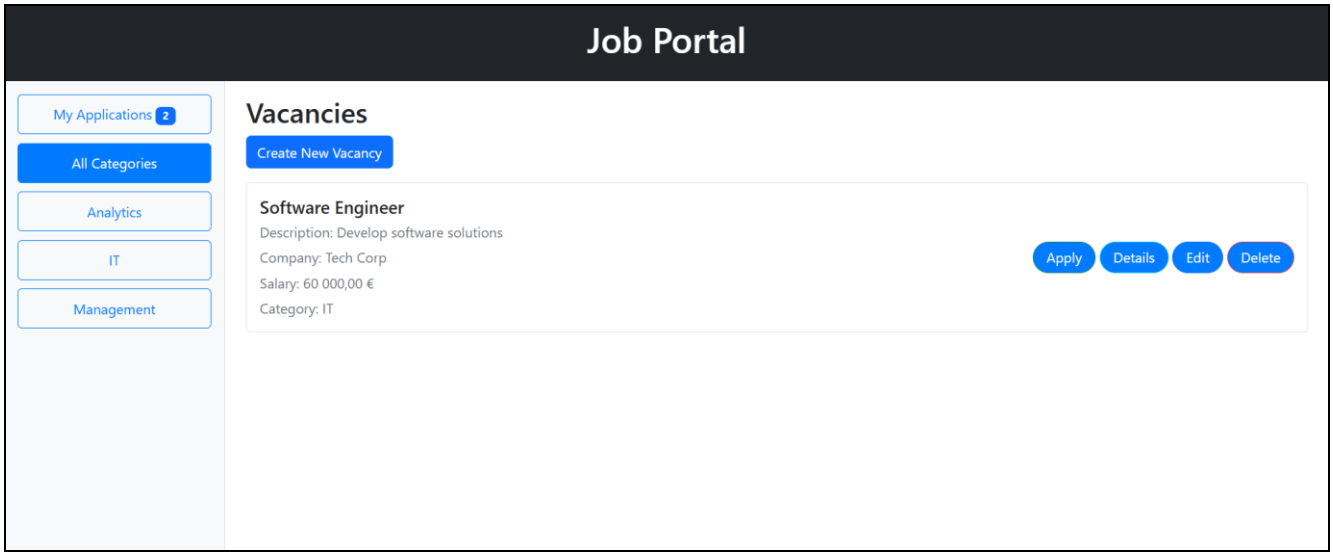

Рис. 8. Результат видалення вакансії.

Лістинг VacancyController.cs:

```
using Microsoft.AspNetCore.Mvc;
using Lab4.Models;
using Microsoft.EntityFrameworkCore;
using System.Text.Json;
using Lab4.Models.ViewModels;

namespace Lab4.Controllers
{
    public class VacancyController : Controller
    {
        private readonly IJobRepository repository;
        private int pageSize = 5;

        public VacancyController(IJobRepository repo)
        {
            repository = repo;
        }

        public IActionResult Index(string category, int page = 1)
```

```
        {
            var vacancies = repository.Vacancies
                .Where(v => category == null || v.Category == category)
                .OrderBy(v => v.VacancyId);

            var totalItems = vacancies.Count();
            var pagedVacancies = vacancies
                .Skip((page - 1) * pageSize)
                .Take(pageSize)
                .Select(v => new VacancyViewModel
                {
                    VacancyId = v.VacancyId,
                    Title = v.Title,
                    Description = v.Description,
                    Company = v.Company,
                    Salary = v.Salary,
                    Category = v.Category
                });

            var viewModel = new VacanciesListViewModel
            {
                Vacancies = pagedVacancies,
                PagingInfo = new PagingInfo
                {
                    CurrentPage = page,
                    ItemsPerPage = pageSize,
                    TotalItems = totalItems
                },
                CurrentCategory = category
            };

            return View("~/Views/Home/Index.cshtml", viewModel);
        }

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Vacancy vacancy)
        {
            if (ModelState.IsValid)
            {
                repository.AddVacancy(vacancy);
                return RedirectToAction("Index");
            }
            return View(vacancy);
        }

        [HttpGet]
        public IActionResult Details(long id)
        {
            var vacancy = repository.Vacancies
                .Include(v => v.Applications)
                .FirstOrDefault(v => v.VacancyId == id);
            if (vacancy == null) return NotFound();
            return View(vacancy);
        }

        [HttpGet]
        public IActionResult Edit(long id)
        {
            var vacancy = repository.Vacancies.FirstOrDefault(v => v.VacancyId == id);
```

```
            if (vacancy == null) return NotFound();
            return View(vacancy);
        }

        [HttpPost]
        public IActionResult Edit(Vacancy vacancy)
        {
            if (ModelState.IsValid)
            {
                repository.UpdateVacancy(vacancy);
                return RedirectToAction("Index");
            }
            return View(vacancy);
        }

        [HttpGet]
        public IActionResult Delete(long id)
        {
            var vacancy = repository.Vacancies.FirstOrDefault(v => v.VacancyId == id);
            if (vacancy == null) return NotFound();
            return View(vacancy);
        }

        [HttpPost, ActionName("Delete")]
        public IActionResult DeleteConfirmed(long id)
        {
            var vacancy = repository.Vacancies.FirstOrDefault(v => v.VacancyId == id);
            if (vacancy == null) return NotFound();
            repository.DeleteVacancy(vacancy);
            return RedirectToAction("Index");
        }

        [HttpGet]
        public IActionResult Apply(long id)
        {
            var vacancy = repository.Vacancies.FirstOrDefault(v => v.VacancyId == id);
            if (vacancy == null) return NotFound();

            var draftKey = $"DraftApplication_{id}";
            var draft = HttpContext.Session.GetString(draftKey);
            var application = draft != null
                ? JsonSerializer.Deserialize<Application>(draft)
                : new Application { VacancyId = id };

            ViewBag.VacancyTitle = vacancy.Title;
            return View(application);
        }

        [HttpPost]
        public IActionResult Apply(Application application)
        {
            Console.WriteLine($"Received: VacancyId={application.VacancyId},
Resume={application.Resume}");

            ModelState.Remove("CandidateId");
            ModelState.Remove("Vacancy");
            ModelState.Remove("AppliedDate");

            if (ModelState.IsValid)
            {
                application.CandidateId = "candidate-id";
                application.AppliedDate = DateTime.Now;
           Console.WriteLine($"Before saving: CandidateId={application.CandidateId},
VacancyId={application.VacancyId}, Resume={application.Resume}");
                repository.AddApplication(application);
```

```
                Console.WriteLine("Application saved successfully");
                var draftKey = $"DraftApplication_{application.VacancyId}";
                HttpContext.Session.Remove(draftKey);
                TempData["Message"] = "Application submitted successfully!";
                return RedirectToAction("MyApplications");
            }

            Console.WriteLine("ModelState is invalid");
            foreach (var error in ModelState.Values.SelectMany(v => v.Errors))
            {
                Console.WriteLine($"Validation error: {error.ErrorMessage}");
            }

var vacancy = repository.Vacancies.FirstOrDefault(v => v.VacancyId ==
application.VacancyId);
            ViewBag.VacancyTitle = vacancy?.Title;
            return View(application);
        }

        [HttpPost]
        public IActionResult SaveDraft(Application application)
        {
            var draftKey = $"DraftApplication_{application.VacancyId}";
       HttpContext.Session.SetString(draftKey, JsonSerializer.Serialize(application));
            return Json(new { success = true });
        }

        [HttpGet]
        public IActionResult MyApplications()
        {
            var applications = repository.Applications
                .Where(a => a.CandidateId == "candidate-id")
                .Include(a => a.Vacancy)
                .Select(a => new ApplicationViewModel
                {
                    ApplicationId = a.ApplicationId,
                  VacancyTitle = a.Vacancy != null ? a.Vacancy.Title : "Vacancy Removed",
                    Resume = a.Resume,
                    AppliedDate = a.AppliedDate
                });
       Console.WriteLine($"Found {applications.Count()} applications for candidate-id");
            if (!applications.Any())
            {
                ViewBag.Message = "You haven't applied to any vacancies yet.";
            }
            return View(applications);
        }

        [HttpGet]
        public IActionResult EditApplication(long id)
        {
var application = repository.Applications.FirstOrDefault(a => a.ApplicationId == id &&
a.CandidateId == "candidate-id");
            if (application == null) return NotFound();

            var draftKey = $"DraftEditApplication_{id}";
            var draft = HttpContext.Session.GetString(draftKey);
            if (draft != null)
            {
                var draftApplication = JsonSerializer.Deserialize<Application>(draft);
                application.Resume = draftApplication.Resume;
            }

            return View(application);
        }
```

```
        [HttpPost]
        public IActionResult EditApplication(Application application)
        {
            ModelState.Remove("CandidateId");
            ModelState.Remove("Vacancy");
            ModelState.Remove("AppliedDate");

            if (ModelState.IsValid)
            {
var existingApplication = repository.Applications.FirstOrDefault(a => a.ApplicationId ==
application.ApplicationId && a.CandidateId == "candidate-id");
                if (existingApplication == null) return NotFound();

                existingApplication.Resume = application.Resume;
                repository.UpdateApplication(existingApplication);
                var draftKey = $"DraftEditApplication_{application.ApplicationId}";
                HttpContext.Session.Remove(draftKey);
                TempData["Message"] = "Application updated successfully!";
                return RedirectToAction("MyApplications");
            }

            return View(application);
        }

        [HttpPost]
        public IActionResult SaveEditDraft(Application application)
        {
            var draftKey = $"DraftEditApplication_{application.ApplicationId}";
HttpContext.Session.SetString(draftKey, JsonSerializer.Serialize(application));
            return Json(new { success = true });
        }

        [HttpPost]
        public IActionResult DeleteApplication(long id)
        {
var application = repository.Applications.FirstOrDefault(a => a.ApplicationId == id &&
a.CandidateId == "candidate-id");
            if (application == null) return NotFound();

            repository.DeleteApplication(application);
            TempData["Message"] = "Application deleted successfully!";
            return RedirectToAction("MyApplications");
        }

        [HttpGet]
        public IActionResult ApplicationDetails(long id)
        {
            var application = repository.Applications
                .Include(a => a.Vacancy)
.FirstOrDefault(a => a.ApplicationId == id && a.CandidateId == "candidate-id");
            if (application == null) return NotFound();
            return View(application);
        }
    }
}
```

Завдання 3.

Додати серверну валідацію для перевірки введених користувачем даних перед виконанням CRUD-операцій. Використати DataAnnotations для реалізації перевірки введених даних.

Лістинг Vacancy.cs:

```
using System.ComponentModel.DataAnnotations;
using System.Collections.Generic;

namespace Lab4.Models
{
    public class Vacancy
    {
        public long VacancyId { get; set; }

        [Required(ErrorMessage = "Please enter a title")]
        [StringLength(100, ErrorMessage = "Title cannot be longer than 100 characters")]
        public string Title { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please enter a description")]
        public string Description { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please enter a company name")]
 [StringLength(50, ErrorMessage = "Company name cannot be longer than 50 characters")]
        public string Company { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please enter a salary")]
        [Range(0.01, double.MaxValue, ErrorMessage = "Please enter a positive salary")]
        public decimal Salary { get; set; }

        [Required(ErrorMessage = "Please specify a category")]
        [StringLength(50, ErrorMessage = "Category cannot be longer than 50 characters")]
        public string Category { get; set; } = string.Empty;

        public ICollection<Lab4.Models.Application> Applications { get; set; } = new
List<Lab4.Models.Application>();

    }
}
```

Лістинг Application.cs:

```
using System.ComponentModel.DataAnnotations;

namespace Lab4.Models
{
    public class Application
    {
        public long ApplicationId { get; set; }

        [Required(ErrorMessage = "Candidate ID is required")]
        public string CandidateId { get; set; } = string.Empty;

        public long VacancyId { get; set; }

        [Required(ErrorMessage = "Please enter a resume")]
      [StringLength(1000, ErrorMessage = "Resume cannot be longer than 1000 characters")]
        public string Resume { get; set; } = string.Empty;

        public DateTime AppliedDate { get; set; }

        public Vacancy Vacancy { get; set; }
    }
}
```

Лістинг (частковий) VacancyController.cs:

```
[HttpPost]
public IActionResult Create(Vacancy vacancy)
```

```
{
    if (ModelState.IsValid)
    {
        repository.AddVacancy(vacancy);
        return RedirectToAction("Index");
    }
    return View(vacancy);
}


        [HttpPost]
        public IActionResult Edit(Vacancy vacancy)
        {
            if (ModelState.IsValid)
            {
                repository.UpdateVacancy(vacancy);
                return RedirectToAction("Index");
            }
            return View(vacancy);
        }


[HttpPost]
        public IActionResult Apply(Application application)
        {
            Console.WriteLine($"Received: VacancyId={application.VacancyId},
Resume={application.Resume}");

            ModelState.Remove("CandidateId");
            ModelState.Remove("Vacancy");
            ModelState.Remove("AppliedDate");

            if (ModelState.IsValid)
            {
                application.CandidateId = "candidate-id";
                application.AppliedDate = DateTime.Now;
           Console.WriteLine($"Before saving: CandidateId={application.CandidateId},
VacancyId={application.VacancyId}, Resume={application.Resume}");
                repository.AddApplication(application);
                Console.WriteLine("Application saved successfully");
                var draftKey = $"DraftApplication_{application.VacancyId}";
                HttpContext.Session.Remove(draftKey);
                TempData["Message"] = "Application submitted successfully!";
                return RedirectToAction("MyApplications");
            }

            Console.WriteLine("ModelState is invalid");
            foreach (var error in ModelState.Values.SelectMany(v => v.Errors))
            {
                Console.WriteLine($"Validation error: {error.ErrorMessage}");
            }

var vacancy = repository.Vacancies.FirstOrDefault(v => v.VacancyId ==
application.VacancyId);
            ViewBag.VacancyTitle = vacancy?.Title;
            return View(application);
        }

[HttpPost]
        public IActionResult EditApplication(Application application)
        {
            ModelState.Remove("CandidateId");
            ModelState.Remove("Vacancy");
            ModelState.Remove("AppliedDate");
```

```
            if (ModelState.IsValid)
            {
var existingApplication = repository.Applications.FirstOrDefault(a => a.ApplicationId ==
application.ApplicationId && a.CandidateId == "candidate-id");
                if (existingApplication == null) return NotFound();

                existingApplication.Resume = application.Resume;
                repository.UpdateApplication(existingApplication);
                var draftKey = $"DraftEditApplication_{application.ApplicationId}";
                HttpContext.Session.Remove(draftKey);
                TempData["Message"] = "Application updated successfully!";
                return RedirectToAction("MyApplications");
            }

            return View(application);
        }
```

У представленнях (Create.cshtml, Edit.cshtml, Apply.cshtml, EditApplication.cshtml) додано валідацію через `<span asp-validation-for="Title" class="text-danger"></span>` і підключено

```
@section Scripts
{
    <partial name="_ValidationScriptsPartial" />
}
```

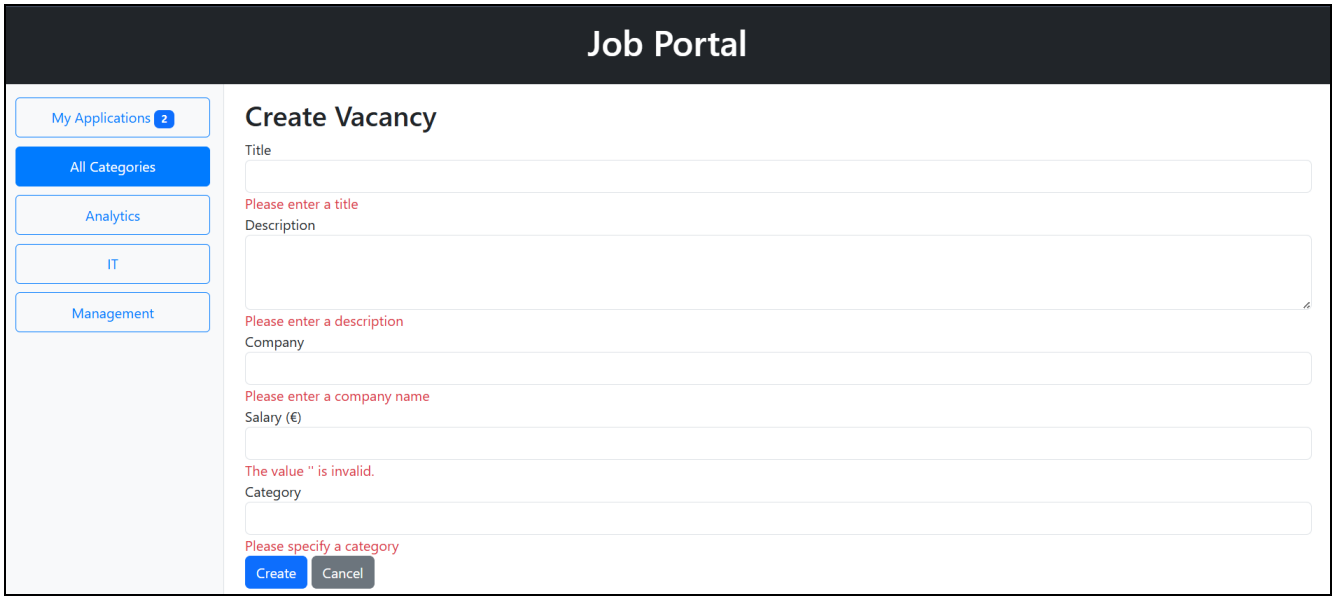

Рис. 9. Валідація введених даних при створенні вакансії.

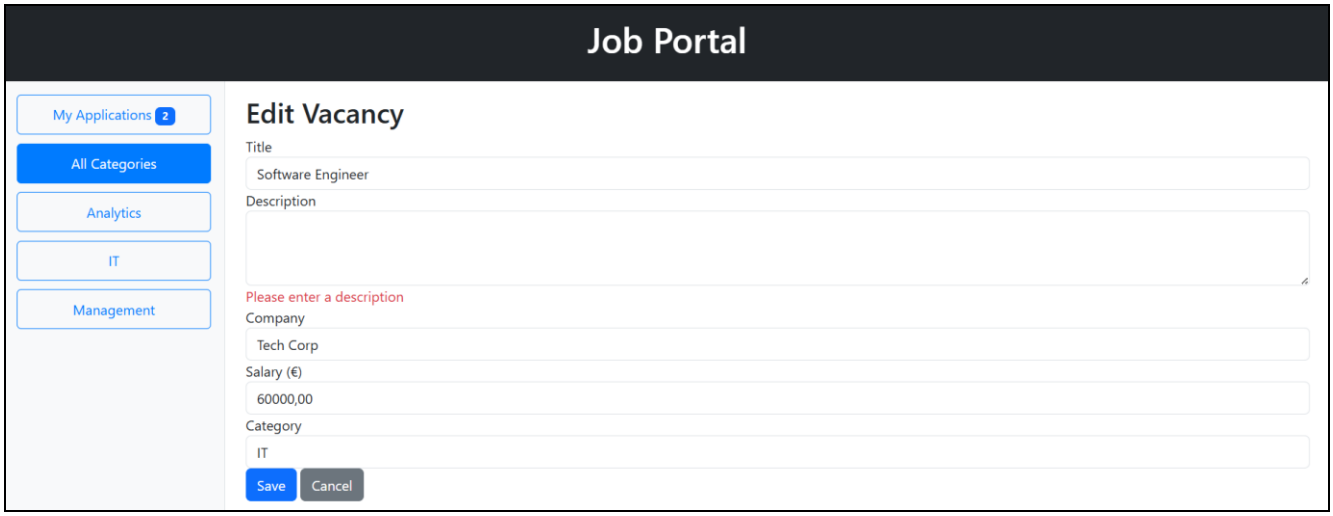

Рис. 10. Валідація введених даних при редагуванні вакансії.

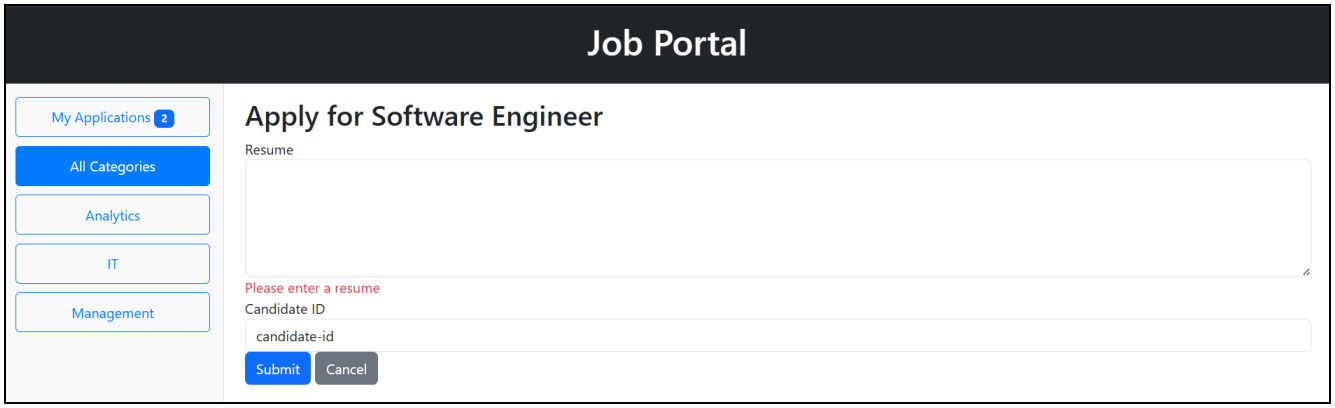

Рис. 11. Валідація введених даних при подачі заявки.

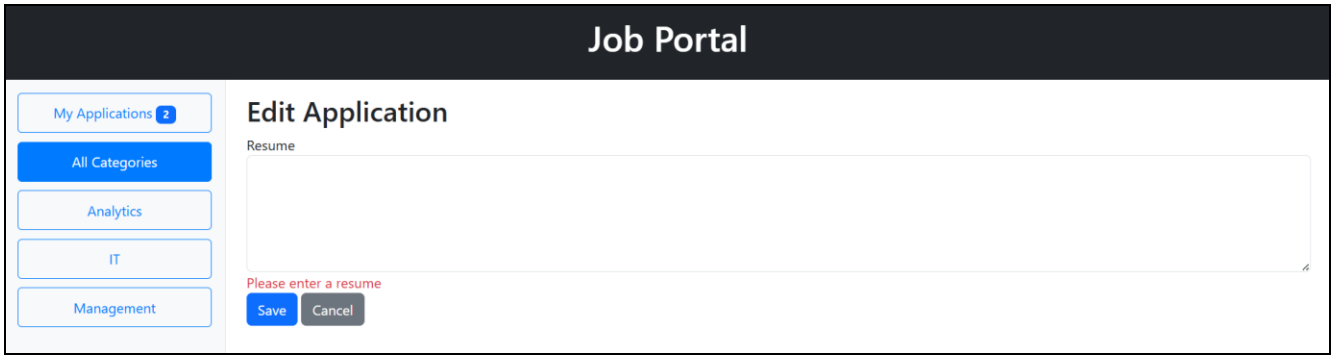

Рис. 12. Валідація введених даних при редагуванні заявки.

Посилання на репозиторій:

[Lab4 · main · IPZ / 2023-2027 / IPZ-23-5 / Bubon Lina / ASP.NET · GitLab]